Application of Markov Chain and Computer Vision

NPC behavior is driven by a Markov chain model, where each character transitions between states—idle, walk, run, and congregate—based on probabilistic rules. Rather than scripted paths, NPCs respond dynamically to their current state and environment. For example, an NPC in the **"idle" state outside a honky-tonk may have a high probability of transitioning to "congregate"** if other NPCs are nearby, or to "walk" if the surrounding area becomes less populated. This system creates fluid, unscripted crowd behavior that mimics the spontaneous social flow of a real night on Broadway. The use of a **Markov model adds complexity and unpredictability, enhancing realism and immersion** in the virtual Nashville experience.

The game leverages **computer vision** techniques to detect bright regions within the environment and simulate the vibrant, flashing neon light atmosphere characteristic of Broadway. By analyzing **pixel intensity** in real time, the system identifies high-luminance areas—such as signs, marquees, and streetlights—and **dynamically enhances their glow and flicker to mimic the visual energy of the strip.** These light sources are further animated with varying color temperatures and pulse frequencies to create a rhythmic, immersive lighting environment. This approach not only adds visual realism but also reinforces the sensory identity of the setting, making the virtual Nashville feel alive with movement and nightlife.

Partial Code for NPC Interaction

```
def congregate(self,app):
    x = app.congregateSpot[0]
    y = app.congregateSpot[1]
    dx = 0
    dy = 0
    for people in app.peopleCluster:
        d = getDistance(x,y,people.x,people.y)
        if d<=50:
            dx = -1 if people.x>x else 1
            dy = -1 if people.y>y else 1
            people.x += dx
            people.y += dy
        else:
            self.run(app,1)
```

```
def idle(self,app):
    self.x = self.x
    self.y = self.y
```

```
def walk(self,app,factor):
    AveragePeople.run(self,app,1)
```

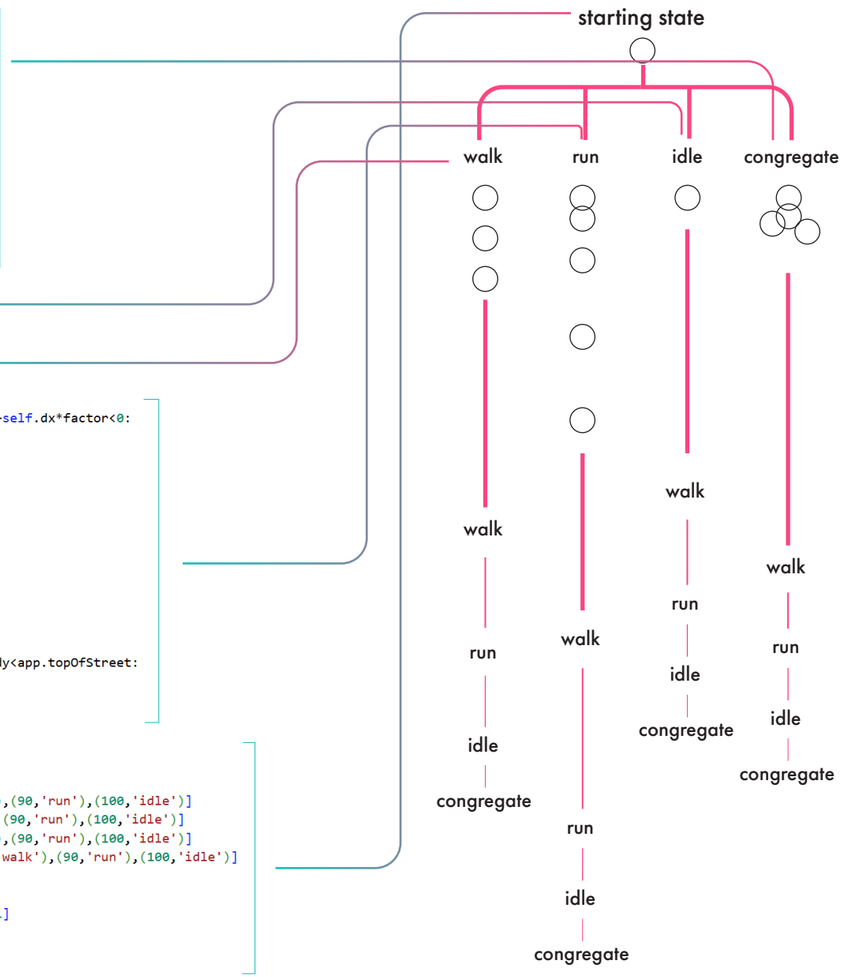
```
def run(self,app,factor):
    if self.x+self.dx*factor>app.width or self.x+self.dx*factor<0:
        self.dx *= -1

    answer = self.getClosest(app)
    if answer != None:
        closestPx,closestPy = answer
        if self.y>closestPy:
            self.dy = random.randint(3,5)
        else:
            self.dy = random.randint(3,5)*-1
    else:
        self.dy = 0

    if self.y+self.dy>app.height or self.y+self.dy<app.topOfStreet:
        self.dy *= -1
    self.x += self.dx*factor
    self.y += self.dy*factor
```

```
def chooseAction(self):
    num = random.randint(0,100)
    prob = dict()
    prob['walk'] = [(-5,'congregate'),(70,'walk'),(90,'run'),(100,'idle')]
    prob['run'] = [(-5,'congregate'),(50,'walk'),(90,'run'),(100,'idle')]
    prob['idle'] = [(-5,'congregate'),(70,'walk'),(90,'run'),(100,'idle')]
    prob['congregate'] = [(-5,'congregate'),(80,'walk'),(90,'run'),(100,'idle')]
    listAction = prob[self.currAction]
    for action in listAction:
        probablity, activity = action[0],action[1]
        if num<=probablity:
            self.currAction = activity
            return
```

Diagram of NPC Interaction



Regular Scene

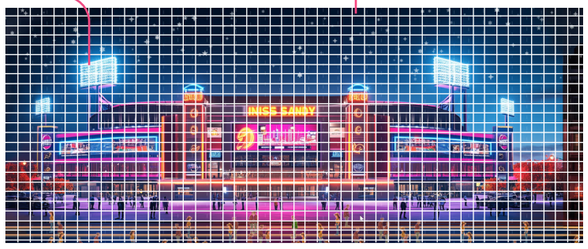
```
def url_to_image(url,readFlag = cv.IMREAD_COLOR):
    resp = urlopen(url)
    image = np.asarray(bytearray(resp.read()),dtype='uint8')
    image = cv.imdecode(image,readFlag)
    return image

app.img = url_to_image(app.url)
app.img = cv.medianBlur(app.img,5)
ret,th1 = cv.threshold(app.img,150,255,cv.THRESH_BINARY)
app.coordinates = np.argwhere(th1)
app.sampleSize = len(th1)*0.05
print(f'smaplesize: {app.sampleSize}')

app.rows = 50
app.cols = app.rows*2
app.board = [[None]*app.cols for row in range(app.rows)]
app.board2 = app.board
app.setMaxShapeCount(4000)

sortCoordinate(app)
```

The scene was segmented into discrete cells, and the cumulative **RGB intensity** of each cell was computed. Leveraging the OpenCV library, an **adaptive thresholding** technique was applied to isolate **high-intensity regions**, producing a visual effect reminiscent of Broadway's **flashing neon signage.**



Computer Vision Finding Bright Spots(original pic top, Opencv enhancement pic bot)